

Structure de données

Listes Chainées Simples

DR. HASNA ELALAOUI

h.elalaoui@uca.ac.ma

Listes chaînées simples

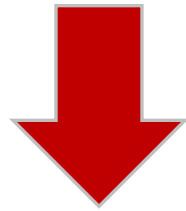
Détails sur la partie

- ❑ Contexte général
 - ❑ Problématique
 - ❑ Présentation générale
 - ❑ Caractéristiques
- ❑ Manipulation des listes chaînées
 - ❑ Structure de contrôle
 - ❑ Fonctions de gestion d'une liste chaînée
- ❑ Autres types de listes

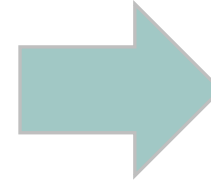
Contexte Général

Problématique

`int tab[5] = {1,2,3,4,5}`



1
2
3
4
5



Problèmes

- ☐ Capacité limitée
- ☐ Ajout/ Suppression quasi-impossible
- ☐ Emplacement successif

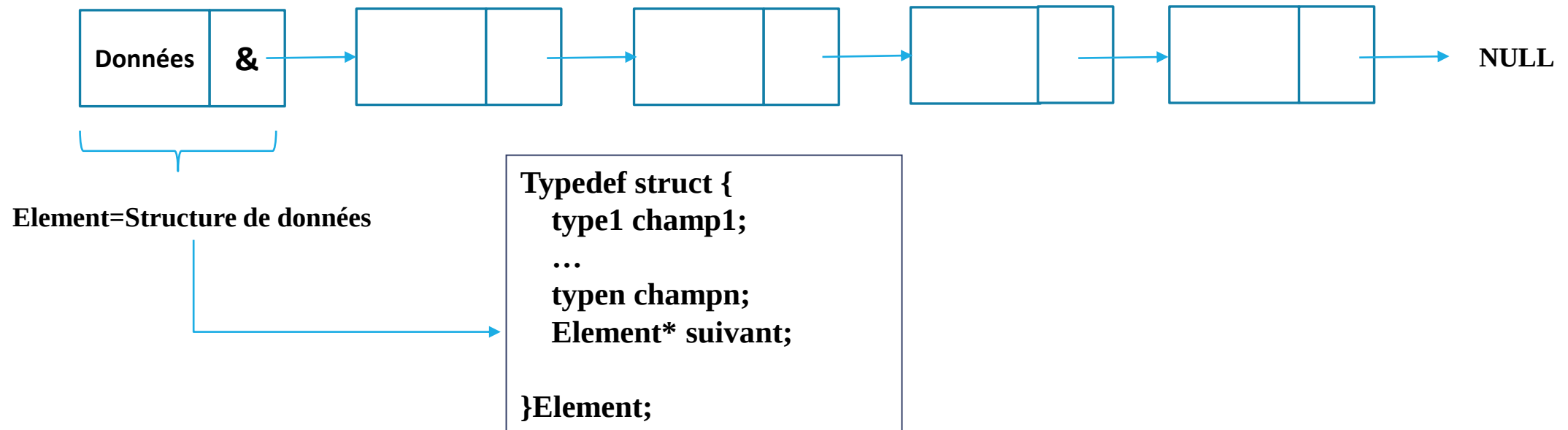
Solution

Besoin d'une structure dynamique

Liste Chainée

Présentation générale

- ❑ Une liste chaînée est un ensemble fini d'éléments enchainés.
- ❑ Une liste chaînée est une structure de données linéaire, dans laquelle les éléments **ne sont pas** stockés à des emplacements de mémoire contigus.
- ❑ Chaque élément est appelé **noeud** ou **maillon**.



Caractéristiques

- ❑ Chaque élément est constitué de 2 parties:
 - ❑ chaque élément de la liste peut contenir ce que l'on désire (Partie **données**)
 - ❑ chaque élément possède un pointeur vers l'élément qui le suit (Partie **adresse** ou **pointeur**)
- ❑ S'il n'y a pas de maillon suivant, le pointeur vaut **NULL**.
- ❑ Elle ne perd pas d'espace mémoire.
- ❑ L'accès aléatoire n'est pas autorisé dans une liste chaînée. Nous devons accéder séquentiellement aux éléments à partir du premier nœud.

Liste vs. Tableau

- ❑ Dans un tableau les données sont stockées à des emplacements de mémoire contigus. Contrairement à une liste chaînée.
- ❑ Contrairement au tableau, la liste n'est pas allouée en une seule fois, mais chaque élément est alloué indépendamment (maillon par maillon).
- ❑ La liste chaînée offre une facilité d'insertion / suppression surtout en cas d'éléments triés. Contrairement au tableau où nous devons insérer et déplacer le reste des éléments.
- ❑ Une liste chaînée est une structure de données qui change durant l'exécution. Elle peut augmenter ou diminuer en taille lors de l'exécution d'un programme.

Exemple

- En revenant à l'exemple du tableau à 5 éléments, il pourra être schématisé avec une liste chaînée comme suit:



```
typedef struct {  
    int nombre;  
    Element* suivant;  
} Element;
```

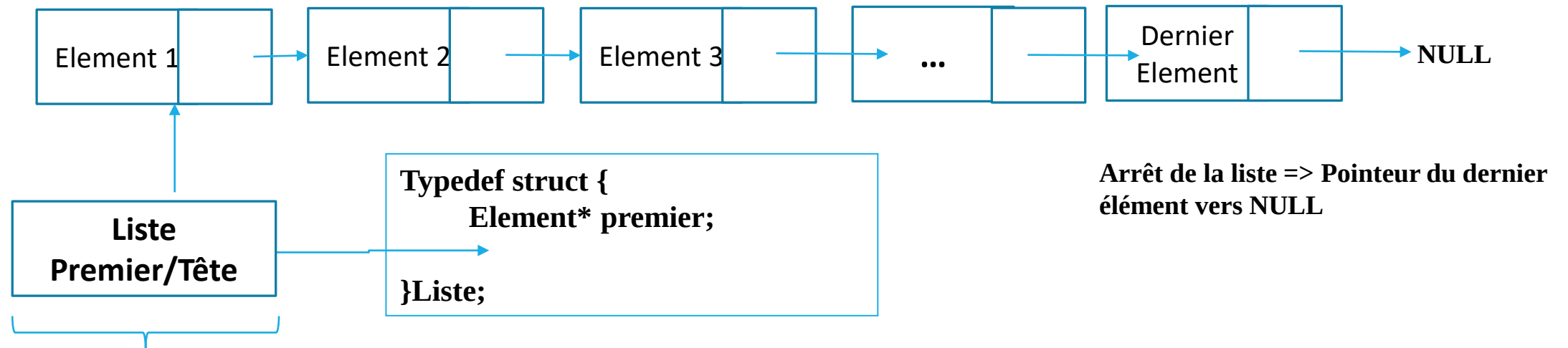


Comment peut-on contrôler cette liste?

Manipulation des listes chaînées

Etape 1 : Création de la structure de contrôle

- ✓ En plus de la structure qu'on vient de créer (structure représentant chaque élément/nœud de la liste), on doit créer une 2^{ème} structure appelée **structure de contrôle**.
- ✓ Il est **PRIMORDIAL** de créer la structure de contrôle: c'est-à-dire un élément qui va contrôler toute la chaîne.
- ✓ C'est un élément qui conserve l'adresse du **premier** élément de la liste.



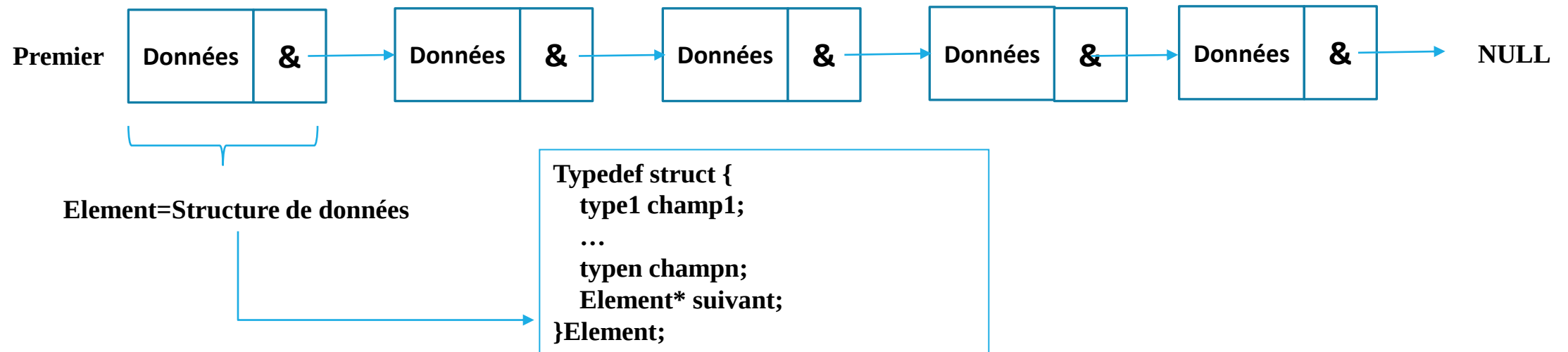
L'adresse du premier élément de la liste

Etape 2: Fonctions de gestion

- ❑ Nous avons créé deux structures qui permettent de gérer une liste chaînée:
 - ❑ **Element**, qui correspond à un élément de la liste et que l'on peut dupliquer **autant de fois** que nécessaire
 - ❑ **Liste**, qui contrôle l'ensemble de la liste (**en un seul exemplaire**)
- ❑ Reste maintenant à créer des fonctions essentielles à la gestion de la liste chaînée
 1. Initialisation de la liste
 2. Ajout d'un élément: Au début, au milieu et à la fin (Faire attention à l'ordre des opérations)
 3. Suppression d'un élément: Au début, au milieu et à la fin (Faire attention à l'ordre des instructions)
 4. Recherche d'un élément suivant un critère
 5. Modification d'un élément de la liste

Initialisation – Principe

- ❑ Définition de la structure de données
- ❑ Deux choix sont possibles: soit on crée **une liste vide**, soit on **boucle sur le nombre d'éléments** initiaux de la liste
- ❑ Initialiser le premier élément de la liste (premier) = allocation de la mémoire
- ❑ Si 2^{ème} choix, saisir les données pour chaque élément et passer à l'élément suivant (sans oublier l'allocation dynamique)
- ❑ Ne pas oublier l'arrêt de la liste.



Initialisation - Code

Liste vide

```
Liste* initialisation() {  
    Liste* liste = malloc(sizeof(Liste));  
    Element* element= malloc(sizeof(Element));  
    //initialisation de la partie données  
    element->nombre = 0;  
  
    //initialisation de la partie pointeur  
    element->suivant = null;  
    liste->premier = element;  
    return liste;  
}
```

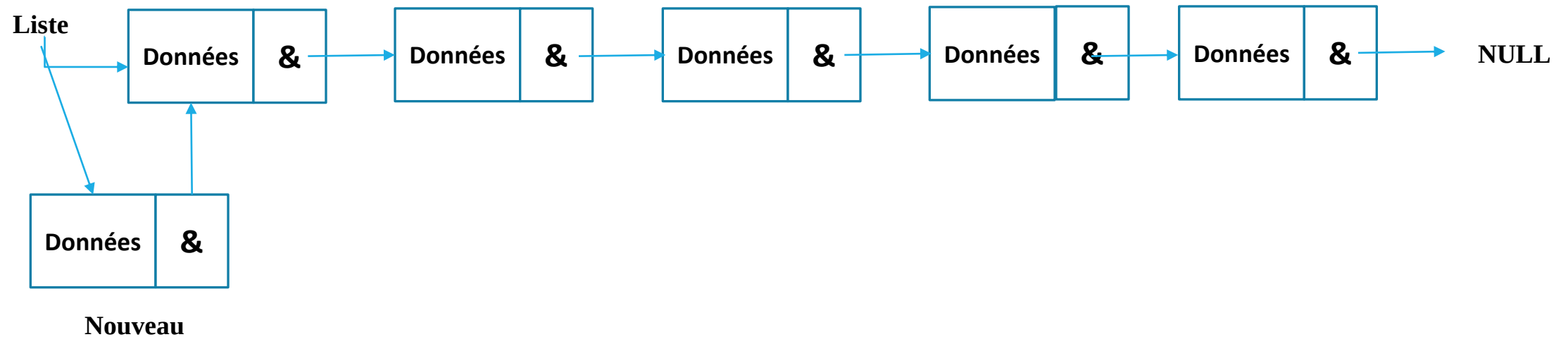
Liste avec des éléments

```
Liste* initialisation(int taille) {  
    Liste* liste = malloc(sizeof(Liste));  
    Element* element= malloc(sizeof(Element));  
    liste->premier=element;  
    for(int i=1;i<=taille;i++){  
        //initialisation de la partie données  
        scanf("%d", &element->nombre);  
  
        //initialisation de la partie pointeur  
        if(i==taille) element->suivant=NULL;  
        else{  
            element->suivant=malloc(sizeof(Element));  
            element=element->suivant;  
        }  
    }  
    return liste;  
}
```

Insertion au début – Principe

- ❑ Initialiser le nouvel élément à insérer (allocation dynamique)
- ❑ Saisie/attribution des données au nouvel élément
- ❑ Pointer le suivant du nouvel élément vers le premier élément de la liste
- ❑ Le nouvel élément devient la tête de la liste.

Ordre à respecter sinon la liste est perdue.



Insertion au début - Code

Insertion au début de la liste

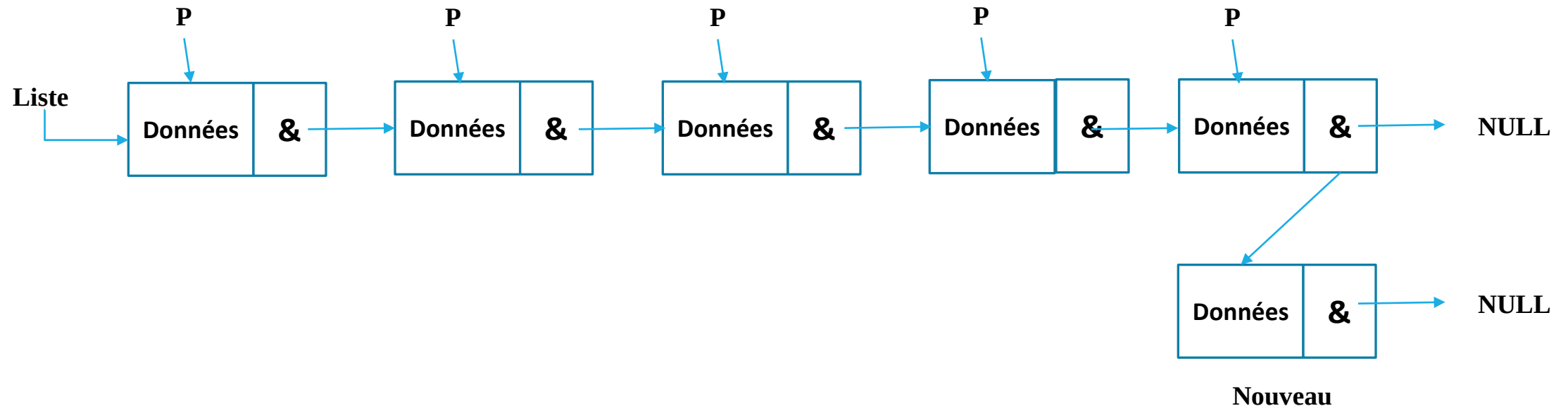
```
void insertionDebut( Liste *liste, int valeur)
{
    //Création du nouvel élément
    Element *nouveau = malloc( sizeof( Element) ) ;

    //Attribution des valeurs à la partie données
    nouveau->nombre = valeur;

    // Insertion de l'élément au début de la liste
    nouveau->suivant = liste->premier;
    liste->premier = nouveau;
}
```

Insertion à la fin – Principe

- ❑ Initialiser l'élément à insérer (allocation dynamique)
- ❑ Saisie/attribution des données au nouvel élément
- ❑ Pointer le suivant du nouvel élément vers NULL
- ❑ Utiliser 1 pointeur parcourir la liste jusqu'à arriver au dernier élément
- ❑ Pointer le dernier élément vers le nouvel élément.



Insertion à la fin – Code

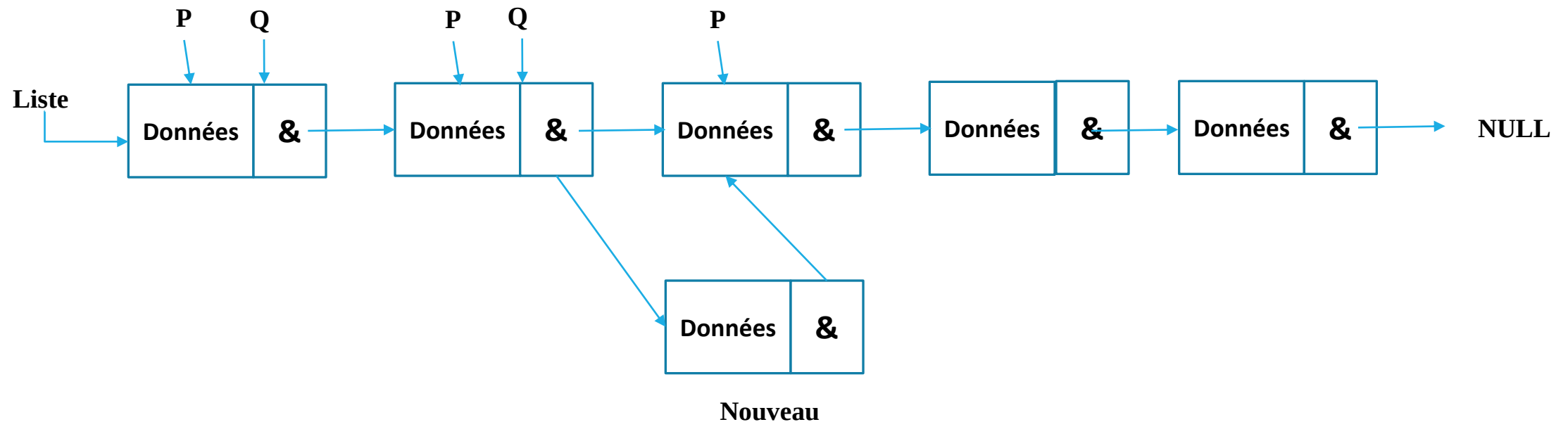
Insertion à la fin de la liste

```
void insertionFin( Liste *liste, int valeur)
{
    Element *p;
    p=liste->premier;
    Element *nouveau = malloc (sizeof(Element));
    nouveau->nombre=valeur;
    nouveau->suivant=NULL;

    while(p->suivant != NULL){
        p=p->suivant;
    }
    p->suivant=nouveau;
}
```

Insertion à une position – Principe

- ❑ Initialiser l'élément à insérer (allocation dynamique)
- ❑ Saisie/attribution des données au nouvel élément
- ❑ Utiliser 2 pointeurs pour avancer 2 par 2 et parcourir la liste jusqu'à arriver à la position souhaitée
- ❑ Pointer l'élément vers le nouvel élément et ce dernier vers l'élément suivant.



Insertion à une position – Code

Insertion à une position

```
void insertionPos( Liste *liste, int valeur, int pos)
{
    int compteur=1;
    Element *p, *q;
    p=liste->premier;
    while(compteur<pos && p!=NULL){
        q=p;
        p=p->suivant;
        compteur++;
    }
```

```
if(p==NULL){
    printf("Insertion impossible\n");
}
else {
    Element *nouveau= malloc(sizeof(Element));
    nouveau->nombre=valeur;
    q->suivant=nouveau;
    nouveau->suivant=p;
}
```

Exercice

- ❑ Sur le même schéma, réaliser les fonctions de gestion suivantes:
 - ❑ Fonction qui calcule et retourne le nombre d'éléments contenus dans la liste.
 - ❑ Fonction d'affichage de la liste.
 - ❑ Fonction de recherche d'un élément suivant un critère. Cette fonction retourne la position de l'élément s'il est trouvé et 0 sinon
 - ❑ Fonction de modification d'un élément par une nouvelle valeur. Cette fonction devra faire appel à la fonction de recherche.